

Fact-based News Verification System

A Hybrid Classification and Evidence-based Verification Prototype

Name	Student ID	Email
Sun Yuchen	A0326248B	e1538079@u.nus.edu
Zhao Jiahui	A0329852U	e1554179@u.nus.edu
Zhang Yuxuan	A0285664N	e1216649@u.nus.edu

Abstract: This report describes a fake-news detection and evidence-based news verification prototype for online news and social media content. The original project proposal focused on offline text analysis: a BERT fake/real classifier, topic modeling, sentiment analysis, and a Streamlit interface. After project feedback, the design was revised because judging only by writing style or offline textual patterns is not sufficient for factual verification. The implemented system therefore keeps BERT as a baseline signal, but extends the solution with claim extraction, live evidence retrieval, evidence comparison, and article-level verdict aggregation.

Keywords: BERT Classification, Claim Extraction, Evidence Retrieval, Evidence Comparison

Contents

1	The Business Problem	1
2	The Datasets Used	1
2.1	Dataset Plan Compared with Final Implementation	2
3	The Solution Approaches	2
3.1	Design Evolution from the Proposal	2
3.2	Baseline Fake/Real Classification	2
3.3	Evidence-based Claim Verification	3
3.4	User Interface	3
4	Test Results Demonstrating System Performance	4
4.1	Evaluation Implications of the Revised Design	5
5	Conclusions	5
6	References to Public Libraries, Models, and Source Code	5
6.1	Public Libraries Used in the Implementation	5
6.2	Models, APIs, and Dataset Resources	6
6.3	Project Source Code Organization	6
7	Individual Contribution of Each Team Member	7

1 The Business Problem

Online news and social media platforms spread information quickly, but users do not always have an efficient way to check whether a news item is factually supported. A misleading article may use credible language while still making claims that are unsupported, contradicted, or unverifiable.

The initial proposal treated fake-news detection mainly as an offline NLP classification problem. In that framing, the system learns textual patterns from labeled examples and predicts whether a new article looks fake or real. This is useful as a screening signal, but it has three important weaknesses:

- it may learn writing style, source patterns, or topic bias instead of verifying factual correctness;
- it cannot explain which claims in the article are true, false, or unsupported;
- it struggles with real-world cases where a well-written article contains a few false factual claims.

Based on this limitation, the business problem was refined from “classify whether the text looks fake” to “help users verify whether important claims in the article are supported by external evidence.” The system should:

- classify whether a news item appears fake or real based on labeled training data;
- extract factual claims from the submitted article;
- retrieve external evidence related to each claim through search;
- compare each claim with retrieved evidence and show whether the evidence supports, refutes, or cannot verify it;
- provide an interpretable article-level verdict rather than only a black-box prediction.

The intended users are students, researchers, journalists, and general readers who need an explainable first-pass news verification tool. The prototype is not designed to replace professional fact-checking, but to demonstrate how machine learning and evidence retrieval can support more transparent verification.

2 The Datasets Used

The repository contains four CSV files under the `data/` directory. These files follow the minimal FakeNewsNet-style structure for the GossipCop and PolitiFact subsets: each row contains a news identifier, article URL, title, and tweet identifiers. In the current implementation, `src/data_loader.py` uses the title column as the primary text source because full article bodies are not included in the local CSV files.

Table 1: Local datasets used by the baseline classifier

File	Rows	Label	Fields
<code>gossipcop_fake.csv</code>	5,323	Fake	<code>id, news_url, title, tweet_ids</code>
<code>gossipcop_real.csv</code>	16,817	Real	<code>id, news_url, title, tweet_ids</code>
<code>politifact_fake.csv</code>	432	Fake	<code>id, news_url, title, tweet_ids</code>
<code>politifact_real.csv</code>	624	Real	<code>id, news_url, title, tweet_ids</code>
Total	23,196	–	Title-level fake/real training data

The class distribution in the local data is imbalanced: 17,441 rows are labeled real and 5,755 rows are labeled fake. The current baseline therefore should be described as a headline or title classifier, not a full article-content classifier. For live evidence retrieval, the system uses the GNews API

rather than a static local evidence corpus.

2.1 Dataset Plan Compared with Final Implementation

The proposal planned to use the Kaggle Fake News Dataset as the primary offline training source and supplement it with 200–500 manually collected examples from fact-checking websites such as Snopes and PolitiFact. That plan was appropriate for the original classifier-centered design. However, after the scope shifted toward search and evidence comparison, the project also needed a retrieval source for current external evidence.

Table 2: Dataset and evidence-source evolution

Stage	Planned or Used Data	Role in the System
Proposal stage	Kaggle fake-news dataset with title, text, and label fields	Primary offline data for training a BERT fake/real classifier.
Proposal stage	Self-collected Snopes and PolitiFact samples	Additional fact-checked examples for diversity and realistic evaluation.
Current repository	GossipCop and PolitiFact CSV files in <code>data/</code>	Local title-level fake/real data for the baseline classifier.
Current repository	GNews API search results	Live external evidence source for claim-level retrieval and comparison.

3 The Solution Approaches

The implemented solution evolved from the original offline NLP plan into a hybrid system with two complementary parts: a baseline supervised classifier and an evidence-based verification pipeline.

3.1 Design Evolution from the Proposal

The proposal contained three offline analysis modules: fake-news classification, topic modeling, and sentiment analysis. The revised design keeps the fake/real classifier, but changes the supporting modules. Topic and sentiment can describe an article, but they do not verify factual claims. For example, an article can have a neutral sentiment and a clear political topic while still containing false numbers, fabricated events, or misleading attributions.

Therefore, the final direction prioritizes search and comparison. The key change is that the system no longer treats the input text alone as the only evidence. Instead, it extracts factual claims, searches for related external news, compares the claim with retrieved evidence, and then explains the final verdict.

3.2 Baseline Fake/Real Classification

The baseline classifier is implemented in the `baseline_model.py` module. It fine-tunes `bert-base-uncased` for binary sequence classification. The data loader combines the news title and cleaned text into a `final_text` field and assigns label 1 to real news and label 0 to fake news. The training code uses an 80/20 train-test split, stratification by label, three training epochs, early stopping, and standard classification metrics: accuracy, precision, recall, and F1.

In the revised system, this classifier is intentionally treated as a baseline rather than the whole decision mechanism. Its role is to estimate whether the article resembles fake or real examples in the offline dataset. The evidence pipeline then checks whether the article’s factual claims can be

Table 3: Original proposal versus revised solution

Component	Original Proposal	Revised / Implemented Direction
Core detection	BERT classifier predicts fake or real from article text.	BERT remains as a baseline prior, but the final verdict also uses claim-level evidence.
Supporting analysis	Topic modeling and sentiment analysis enrich interpretation.	Claim extraction, evidence retrieval, and claim-evidence comparison provide factual support.
Evidence source	Offline labeled dataset only.	Local dataset for baseline training plus GNews search for live evidence.
Explanation	Prediction probability, topic, and sentiment.	Extracted claims, retrieval queries, evidence articles, support/refute/NEI labels, and final aggregation.
Main limitation addressed	Text classification can detect suspicious writing patterns.	Evidence comparison addresses whether concrete factual claims are externally supported.

supported by external sources.

3.3 Evidence-based Claim Verification

The second approach is designed to make the system more explainable. Instead of only returning a fake/real prediction, the application performs claim-level verification:

1. **Text cleaning:** `src/cleaner.py` normalizes user input.
2. **Claim extraction:** `src/claim_extractor.py` splits the article into chunks, filters weak or subjective statements, and selects factual claims containing entities, dates, numbers, or event verbs.
3. **Evidence retrieval:** `src/retriever.py` builds several search queries per claim, calls the GNews API, deduplicates retrieved articles, and ranks evidence using title overlap, text overlap, entity matches, number matches, and API rank.
4. **Claim verification:** `src/verifier.py` assigns one of three labels using token overlap, number consistency, and negation/polarity signals: **supported**, **refuted**, or **not enough information**.
5. **Final aggregation:** `src/aggregator.py` combines claim-level labels and confidence scores into **Likely True**, **Likely False**, or **Unverified**.

3.4 User Interface

The Streamlit application in `app.py` exposes the full workflow. It shows the baseline classifier result, extraction progress, claim-level retrieval queries, best evidence, verification labels, confidence values, and final verdict. This interface is important because the project goal is not only prediction accuracy, but also interpretability.

Table 4: Main implementation modules

Module	Responsibility
<code>app.py</code>	Streamlit interface and end-to-end pipeline controller.
<code>src/data_loader.py</code>	Loads and normalizes GossipCop and PolitiFact CSV files.
<code>src/baseline_model.py</code>	Trains and runs the BERT baseline classifier.
<code>src/claim_extractor.py</code>	Extracts retrieval-ready factual claims from articles.
<code>src/retriever.py</code>	Retrieves and ranks external evidence from GNews.
<code>src/verifier.py</code>	Produces claim-level support/refute/NEI decisions.
<code>src/aggregator.py</code>	Produces the final article-level verdict.

4 Test Results Demonstrating System Performance

The repository contains the code needed to train and evaluate the baseline classifier, but it does not currently include a saved trained model, checkpoint directory, evaluation log, or recorded metric table. Therefore this report should not claim a numerical accuracy, precision, recall, or F1 score until the training script has been run and its outputs have been saved.

Table 5: Current test evidence available in the repository

Test or Evidence	Current Status	Interpretation
Dataset inspection	Completed	The four local CSV files contain 23,196 title-level records with fake/real labels.
Baseline metrics	Pending	<code>src/baseline_model.py</code> computes accuracy, precision, recall, and F1, but no saved evaluation output is present in the repository.
End-to-end verification	Partially implemented	The Streamlit app wires cleaning, baseline prediction, claim extraction, retrieval, verification, and aggregation. Runtime quality depends on a trained baseline model and a valid <code>GNEWS_API_KEY</code> .
Report build check	Passed	The LaTeX report compiles successfully into <code>report/report.pdf</code> .

After model training, the performance section should be updated with the following metrics:

Table 6: Baseline classifier result table to fill after training

Model	Accuracy	Precision	Recall	F1
BERT title classifier	Pending	Pending	Pending	Pending

For the evidence-based verification pipeline, a stronger evaluation would require manually labeled claim-evidence pairs. The recommended metrics are claim-label accuracy, support/refute/NEI macro-

F1, evidence retrieval precision at top k , and article-verdict agreement with human fact-checking labels.

4.1 Evaluation Implications of the Revised Design

The proposal evaluation plan focused on classifier metrics such as accuracy, precision, recall, F1, and confusion matrices. Those metrics remain necessary for the BERT baseline, but they are no longer sufficient for the full system. Because the system now performs search and evidence comparison, evaluation should also check whether retrieved evidence is relevant and whether the claim-level labels are correct.

A complete evaluation should therefore include:

- **Baseline classification performance:** accuracy, precision, recall, F1, and confusion matrix on held-out title/article examples.
- **Claim extraction quality:** whether extracted claims are factual, specific, non-duplicate, and searchable.
- **Retrieval quality:** whether top-ranked GNews results are actually related to the claim.
- **Comparison quality:** whether the verifier correctly assigns supported, refuted, or not-enough-information labels.
- **End-user usefulness:** whether the displayed evidence makes the final verdict understandable to users.

5 Conclusions

The project implements a practical and explainable prototype for fake-news detection and factual verification. The baseline BERT classifier provides a fast fake/real signal from title-level labeled data, while the claim-level pipeline improves transparency by showing extracted claims, retrieval queries, evidence snippets, verification labels, and the reasoning behind the final verdict.

Compared with the proposal-stage design, the final system better addresses the project feedback that a purely offline fake-news classifier is incomplete. Offline classification can identify suspicious writing patterns, but factual verification requires checking claims against external information. The revised system therefore moves from style-based detection toward evidence-based verification.

The main conclusion is that the system design is more informative than a single black-box classifier, but the current repository should be presented as an implemented prototype rather than a fully validated production system. The next required step is to run baseline training, save the evaluation metrics, and create a manually labeled claim-evidence test set for the verification pipeline.

6 References to Public Libraries, Models, and Source Code

This project uses public Python libraries, pretrained models, public dataset resources, and public API documentation. The items below separate resources used directly in the current implementation from resources referenced in the original proposal.

6.1 Public Libraries Used in the Implementation

1. **Streamlit.** Used to build the interactive web interface in `app.py`. Official documentation: Streamlit documentation. Public source code: `streamlit/streamlit`.
2. **PyTorch.** Used as the deep learning runtime for BERT training and inference in `src/baseline_model.py`. Official documentation: PyTorch documentation. Public source code:

pytorch/pytorch.

3. **Hugging Face Transformers**. Used for BERT tokenization, sequence-classification model loading, training configuration, the Trainer API, and the optional text-generation pipeline in the claim extractor. Official documentation: Transformers documentation. Public source code: huggingface/transformers.
4. **Hugging Face Datasets**. Used to convert pandas data frames into Trainer-compatible datasets for baseline model training. Official documentation: Datasets documentation. Public source code: huggingface/datasets.
5. **scikit-learn**. Used for stratified train-test splitting and classification metrics including accuracy, precision, recall, and F1. Official user guide: scikit-learn user guide. Public source code: scikit-learn/scikit-learn.
6. **pandas**. Used for CSV loading, dataset normalization, and tabular preprocessing. Official documentation: pandas documentation. Public source code: pandas-dev/pandas.
7. **NumPy**. Listed as a project dependency and commonly used by the scientific Python stack that supports model training and metrics. Official documentation: NumPy documentation. Public source code: numpy/numpy.
8. **Requests**. Used by `src/retriever.py` to call the GNews search endpoint. Official documentation: Requests documentation. Public source code: psf/requests.
9. **python-dotenv**. Used in `config.py` to load `GNEWS_API_KEY` and other environment variables from a local `.env` file. Public source code and documentation: theskumar/python-dotenv.

6.2 Models, APIs, and Dataset Resources

1. **bert-base-uncased**. Used as the pretrained baseline encoder for binary fake/real sequence classification. Hugging Face model card: Hugging Face **bert-base-uncased** model card. Original BERT public source code and model release: google-research/bert.
2. **GNews API**. Used as the live evidence source for search-based claim verification. The implementation calls the GNews search endpoint and re-ranks returned articles locally. Official API documentation: GNews API documentation.
3. **FakeNewsNet**. The local GossipCop and PolitiFact CSV files follow the minimal FakeNewsNet-style public dataset structure: `id`, `news_url`, `title`, and `tweet_ids`. Public repository: KaiD-MML/FakeNewsNet.
4. **Kaggle Fake and Real News Dataset**. Referenced in the original proposal as a planned offline training dataset, but not present in the current repository implementation. Public dataset page: Kaggle Fake and Real News Dataset.

6.3 Project Source Code Organization

The project implementation itself is organized into the following local source files:

- `app.py`: Streamlit interface and end-to-end runtime controller.
- `config.py`: shared paths, model settings, GNews endpoint, and environment-variable loading.
- `src/data_loader.py`: CSV loading and label normalization.
- `src/baseline_model.py`: BERT baseline training and prediction.
- `src/claim_extractor.py`: claim extraction and claim selection.
- `src/retriever.py`: GNews query generation, retrieval, deduplication, and evidence ranking.
- `src/verifier.py`: claim-evidence comparison and support/refute/NEI labeling.
- `src/aggregator.py`: article-level final verdict aggregation.

7 Individual Contribution of Each Team Member

The proposal identifies three team members: Sun Yuchen, Zhao Jiahui, and Zhang Yuxuan. The repository does not include a detailed contribution log, so the following contribution summary should be reviewed and adjusted by the team before final submission.

Table 7: Team contribution summary

Team Member	Main Responsibility	Contribution Details
Sun Yuchen	Business problem, documentation, and evaluation framing	Defined the project motivation, organized the report around the required sections, summarized limitations, and prepared the evaluation discussion.
Zhao Jiahui	Dataset preparation and baseline model	Prepared the GossipCop/PolitiFact data pipeline, cleaned text fields, configured the BERT baseline classifier, and documented classification metrics to be collected.
Zhang Yuxuan	Evidence-based verification and system integration	Implemented or integrated claim extraction, GNews retrieval, claim verification, verdict aggregation, and the Streamlit interface.